

# Chapter 6

## Transiciones

Por: Aucencia Emeterio-Lara, Ernesto Mujica, Elaine González y Raymond L. Tremblay

Las transiciones describen los cambios individuales entre estados demográficos —como supervivencia, crecimiento y regresión— que conectan el ciclo de vida con la dinámica poblacional. Este capítulo introduce el concepto de transiciones como los procesos fundamentales que determinan la estructura y evolución de una población a lo largo del tiempo.

### 6.1 Transiciones demográficas y construcción del ciclo de vida poblacional

Las transiciones representan los procesos mediante los cuales los individuos cambian de estado entre intervalos de tiempo, ya sea permaneciendo en el mismo estadio, creciendo a uno nuevo, retrocediendo o contribuyendo a la población a través de la reproducción. En los modelos de proyección poblacional, estas transiciones constituyen los elementos básicos de la matriz, conectando las tasas vitales individuales con la dinámica poblacional a nivel agregado.

En este capítulo se analizan las transiciones como componentes explícitos del ciclo de vida, mostrando cómo su definición y parametrización influyen directamente en la estructura de la matriz de proyección y en la dinámica resultante. Se discute cómo diferentes formulaciones de las transiciones reflejan supuestos biológicos distintos, tales como crecimiento continuo versus discreto, estasis, o la incorporación de retrocesos ontogenéticos.

A diferencia de los capítulos sobre crecimiento poblacional o propiedades matriciales, este capítulo se centra en el nivel **causal** de la dinámica demográfica: los procesos que generan los patrones observados de crecimiento, estabilidad o declive. Comprender las transiciones es esencial para interpretar análisis posteriores de sensibilidad, elasticidad, proyecciones poblacionales y modelos bayesianos, ya que cualquier cambio en estas métricas surge directamente de modificaciones en las transiciones subyacentes.

### 6.1.0.1 Librerías de R requeridas para el siguiente módulo

```
library(tidyverse)
library(gt)
library(popbio)
library(Rcompadre)
library(DiagrammeR)
```

## 6.2 Introducción

En este capítulo se explica cómo calcular las transiciones para la matriz de Lefkovich (transiciones por etapas). La primera parte aborda el cálculo tradicional a mano, mientras que la segunda muestra cómo realizarlo de manera más eficiente utilizando funciones disponibles en paquetes de R.

Los métodos presentados son una adaptación de los propuestos por (Caswell, 2000) y se incluye una aplicación en orquídeas (Tremblay & Hutchings, 2003). Si ya has realizado análisis de matrices de transición, probablemente conozcas el método tradicional y podrías omitir esta sección. Este procedimiento consiste en calcular las proporciones de individuos que transitan de una clase de edad o etapa a otra, pero hacerlo manualmente puede ser tedioso y propenso a errores. Aun así, es recomendable realizarlo al menos una vez para comprender la mecánica del cálculo.

En la segunda parte del capítulo se muestra cómo calcular las transiciones utilizando la función `projection.matrix` del paquete `popbio`, una alternativa más eficiente y menos susceptible a errores que el método manual.

## 6.3 Definición y parametrización de transiciones

A continuación se describen los distintos tipos de transiciones y su implementación en modelos de proyección poblacional.

Las transiciones definen las propiedades estructurales de la matriz, discutidas en el capítulo sobre propiedades poblacionales. La importancia relativa de cada transición se evalúa más adelante mediante el análisis de elasticidad poblacional.

---

## 6.4 Métodos tradicional de calcular las transiciones

El primer paso consiste en construir una tabla de datos con los individuos muestreados en dos periodos de tiempo (por ejemplo, con un intervalo de un año). Supongamos que tenemos

19 individuos de una orquídea con cuatro estados:

- Plantas pequeñas
- Juveniles
- Adultos
- Muerto

En el primer periodo:

- 7 individuos estaban en plantas pequeñas
- 6 en juveniles
- 6 en adultos

Para calcular las transiciones, debemos obtener la proporción de individuos que pasan de un estado a otro. Por ejemplo, de los 7 individuos que eran plantas pequeñas en el primer periodo:

- 1 permaneció como planta pequeña
- 2 pasaron a juveniles
- 3 pasaron a adultos
- 1 murió

Esto se traduce en las siguientes proporciones:

- Plantas pequeñas  $\rightarrow$  Plantas pequeñas:  $\frac{1}{7} = 0.142871$
- Plantas pequeñas  $\rightarrow$  Juveniles:  $\frac{2}{7} = 0.285772$
- Plantas pequeñas  $\rightarrow$  Adultos:  $\frac{3}{7} = 0.428573$
- Plantas pequeñas  $\rightarrow$  Muerto:  $\frac{1}{7} = 0.142871$

La suma de estas proporciones debe ser igual a 1 (o muy cercana, considerando redondeos).

```
a=1/7
b=2/7
c=3/7
d=1/7

sum=a+b+c+d
sum
```

[1] 1

La suma de la proporciones calculado de las proporciones

| Num_ind | anio_1           | anio_2           | transición |
|---------|------------------|------------------|------------|
| 1       | plantas_pequeñas | Muerto           | 0.1428     |
| 2       | plantas_pequeñas | plantas_pequeñas | 0.1428     |
| 2       | plantas_pequeñas | juvenil          | 0.2857     |
| 3       | plantas_pequeñas | adulto           | 0.4285     |
| 4       | juvenil          | Muerto           | 0.1666     |
| 4       | juvenil          | plantas_pequeñas | 0.0000     |
| 5       | juvenil          | juvenil          | 0.5000     |
| 6       | juvenil          | adulto           | 0.3333     |
| 7       | adulto           | Muerto           | 0.3333     |
| 7       | adulto           | plantas_pequeñas | 0.0000     |
| 8       | adulto           | juvenil          | 0.0000     |
| 9       | adulto           | adulto           | 0.6777     |

```
sum2=0.1428+0.2857+0.4285+0.1428
sum2
```

```
[1] 0.9998
```

**Nota:** La suma de todas las transiciones debe ser exactamente 1.000. No puede ser mayor ni menor que 1. Si la suma es distinta de 1, significa que hay un error en el cálculo de las proporciones. Una causa frecuente de este problema es el redondeo excesivo o incorrecto. En este caso, la suma total es 0.9998, lo cual es suficientemente cercano a 1.0.

Ahora calculamos las otras transiciones de juveniles a otras clases

- $0/6 = 0.0000$ , transitaron de “juveniles” a “plantas pequeñas”
- $3/6 = 0.5000$ , transitaron de “juveniles” a “juveniles”
- $2/6 = 0.3333$ , transitaron de “juveniles” a “adultos”
- $1/6 = 0.1666$ , murieron entre el primer y segundo periodo

De adultos a otras clases

- $0/6 = 0$ , transitaron de “adulto” a “plantas pequeñas”
- $0/6 = 0$ , transitaron de “adulto” a “juveniles”
- $4/6 = 0.6667$ , transitaron de “adulto” a “adulto”
- $2/6 = 0.3333$ , murieron antes del segundo periodo

Aquí la lista completa de las transiciones a todas las etapas posibles.

### 6.4.1 Construcción de la matriz de transiciones

Ahora utilizamos los valores calculados anteriormente para construir la matriz de transiciones.

**Importante:**

- Las columnas representan el estado en el primer periodo.
- Las filas representan el estado en el segundo periodo.

Las transiciones hacia el estado “Muerto” no se incluyen en la matriz, ya que los análisis reconocen automáticamente que la proporción de mortalidad se obtiene como:

$$\text{Proporcin de fallecimiento} = 1 - \text{suma de las transiciones en la fila}$$

Para crear la matriz en R, se utiliza la función `matrix()`, que incluye los siguientes argumentos:

- `nrow` = número de filas (y columnas, porque la matriz siempre será cuadrada).
- `byrow` = indica si los datos se llenan por filas (TRUE) o por columnas (FALSE).

```
ficticia_matrix=matrix(c(0.1428, 0.0000, 0.0000,
                        0.2857, 0.5000, 0.0000,
                        0.4285, 0.3333, 0.6777), nrow=3, byrow=TRUE)

ficticia_matrix
```

```
      [,1] [,2] [,3]
[1,] 0.1428 0.0000 0.0000
[2,] 0.2857 0.5000 0.0000
[3,] 0.4285 0.3333 0.6777
```

### 6.4.2 Añadiendo los nombres de las etapas

Es una buena práctica asignar nombres a las filas y columnas de la matriz, ya que esto facilita la interpretación y ayuda a detectar inconsistencias o posibles errores.

Para ello, se utilizan dos funciones en R:

- `rownames()`: asigna nombres a las filas.
- `colnames()`: asigna nombres a las columnas.

```
rownames(ficticia_matrix)<-c("plantas_pequeñas", "juvenil", "adulto")
colnames(ficticia_matrix)<-c("plantas_pequeñas", "juvenil", "adulto")
```

```
ficticia_matrix
```

|                  | plantas_pequeñas | juvenil | adulto |
|------------------|------------------|---------|--------|
| plantas_pequeñas | 0.1428           | 0.0000  | 0.0000 |
| juvenil          | 0.2857           | 0.5000  | 0.0000 |
| adulto           | 0.4285           | 0.3333  | 0.6777 |

---

### 6.4.3 Construcción de la matriz con funciones en R

En la siguiente sección aprenderemos a calcular las transiciones de manera más eficiente utilizando funciones disponibles en un paquete de R. La función principal que emplearemos es `projection.matrix` del paquete `popbio`.

Esta función es mucho más rápida y menos propensa a errores que realizar los cálculos manualmente. En investigaciones reales, la cantidad de datos suele ser mucho mayor que en el ejemplo anterior, y a mayor volumen de datos, mayor es la probabilidad de cometer errores si se calculan las transiciones a mano.

La función `projection.matrix` permite calcular tanto las transiciones como la fecundidad a partir de una tabla que contiene los estados de los individuos en dos periodos de tiempo. Esto la convierte en una herramienta muy útil para obtener resultados precisos y eficientes.

Los principales argumentos de la función `projection.matrix` son los siguientes:

```
projection.matrix(
  transitions, # el nombre de la tabla de datos con los estados de los individuos en dos
  stage = NULL, # el nombre de la columna con los estados en el primer tiempo
  fate = NULL, # el nombre de la columna con los estados en el segundo tiempo
  fertility = NULL, la información sobre la fertilidad
  sort = NULL, # si se quiere ordenar los estados de la historia de vida
  add = NULL,
  stages = NULL, # los nombres de las etapas del modelo en el primer y el segundo muestr
  TF = FALSE # si quiere que la matriz de transiciones y de fertilidad sean separados us
)
```

## 6.5 Salida de la función `projection.matrix`

El output de la función `projection.matrix` es una lista que contiene dos matrices:

- **T**: incluye únicamente los valores de transición entre etapas.
- **F**: incluye únicamente los valores de fertilidad, es decir, la cantidad de individuos que se incorporan a cada clase de edad por reproducción no clonal.

## 6.6 Preparación de los datos antes del análisis

Antes de comenzar con los análisis, es importante renombrar las columnas para evitar problemas con la función `projection.matrix`. En este caso:

- Cambiamos `anio_1` a `stage`.
- Cambiamos `anio_2` a `fate`.

Estos nombres corresponden a las etapas en el primer y segundo periodo de tiempo. El término `fate` se traduce al español como “destino”.

Los datos deben estar organizados en formato de tabla, donde:

- Cada fila representa un individuo.
- Cada columna representa una variable.

En nuestro ejemplo, la tabla contiene 19 filas y 4 columnas. La columna “transición” no es necesaria para la función `projection.matrix`.

```
library(gt)
Orchis_ficticia_por_ind=tribble(~Num_ind, ~anio_1, ~anio_2, ~fertilidad,
  1, "plantas_pequeñas", "Muerto", 0,
  2, "plantas_pequeñas", "plantas_pequeñas", 0,
  3, "plantas_pequeñas", "juvenil", 0,
  4, "plantas_pequeñas", "juvenil", 0,
  5, "plantas_pequeñas", "adulto", 0,
  6, "plantas_pequeñas", "adulto", 0,
  7, "plantas_pequeñas", "adulto", 0,
  8, "juvenil", "Muerto", 0,
  9, "juvenil", "juvenil", 0,
  10, "juvenil", "juvenil", 0,
  11, "juvenil", "juvenil", 0,
  12, "juvenil", "adulto", 0,
  13, "juvenil", "adulto", 0,
  14, "adulto", "Muerto", 0,
  15, "adulto", "Muerto", 0,
  16, "adulto", "adulto", 0,
  17, "adulto", "adulto", 1,
  18, "adulto", "adulto", 0,
```

| Num_ind | anio_1           | anio_2           | fertilidad |
|---------|------------------|------------------|------------|
| 1       | plantas_pequeñas | Muerto           | 0          |
| 2       | plantas_pequeñas | plantas_pequeñas | 0          |
| 3       | plantas_pequeñas | juvenil          | 0          |
| 4       | plantas_pequeñas | juvenil          | 0          |
| 5       | plantas_pequeñas | adulto           | 0          |
| 6       | plantas_pequeñas | adulto           | 0          |
| 7       | plantas_pequeñas | adulto           | 0          |
| 8       | juvenil          | Muerto           | 0          |
| 9       | juvenil          | juvenil          | 0          |
| 10      | juvenil          | juvenil          | 0          |
| 11      | juvenil          | juvenil          | 0          |
| 12      | juvenil          | adulto           | 0          |
| 13      | juvenil          | adulto           | 0          |
| 14      | adulto           | Muerto           | 0          |
| 15      | adulto           | Muerto           | 0          |
| 16      | adulto           | adulto           | 0          |
| 17      | adulto           | adulto           | 1          |
| 18      | adulto           | adulto           | 0          |
| 19      | adulto           | adulto           | 0          |

| Num_ind | stage            | fate             | fertilidad |
|---------|------------------|------------------|------------|
| 1       | plantas_pequeñas | Muerto           | 0          |
| 2       | plantas_pequeñas | plantas_pequeñas | 0          |
| 3       | plantas_pequeñas | juvenil          | 0          |
| 4       | plantas_pequeñas | juvenil          | 0          |
| 5       | plantas_pequeñas | adulto           | 0          |
| 6       | plantas_pequeñas | adulto           | 0          |

```

19, "adulto", "adulto",0
)
gt(Orchis_ficticia_por_ind)

```

A continuación, mostramos cómo cambiar los nombres de las columnas a **stage** y **fate**, y luego visualizar los primeros seis individuos de la tabla de datos:

```

Orchis_ficticia_por_ind=Orchis_ficticia_por_ind %>% rename(stage=anio_1, fate=anio_2)
gt(head(Orchis_ficticia_por_ind))

```

Una vez que hemos asignado los nombres a las columnas, podemos utilizar la función `projection.matrix` para calcular las transiciones y la fertilidad.



En este contexto, la columna “fertilidad” representa la cantidad de individuos que se incorporan a cada clase de edad en el siguiente periodo.

En nuestro caso, solo los adultos pueden reproducirse, añadiendo nuevos individuos a la clase “plantas pequeñas”. La fertilidad se calcula como:

$$\text{Fertilidad} = \frac{1}{6} = 0.1666$$

Esto significa que, en promedio, cada adulto añade un nuevo individuo a la clase “plantas pequeñas” en el segundo periodo de tiempo.

Aquí un ejemplo de la función

```
projection.matrix(Orchis_ficticia_df, stage, fate, fertilidad, stages, TF=TRUE)
```

```
Orchis_ficticia_df = as.data.frame(Orchis_ficticia_por_ind) # Nota que es importante con
stages = c("plantas_pequeñas", "juvenil", "adulto") # Las etapas del modelo en el primer
fate = c("plantas_pequeñas", "juvenil", "adulto", "Muerto") # Las etapas del modelo en e
projection.matrix(Orchis_ficticia_df, stage, fate, fertilidad, stages, TF=TRUE)
```

\$T

|                  | plantas_pequeñas | juvenil   | adulto    |
|------------------|------------------|-----------|-----------|
| plantas_pequeñas | 0.1428571        | 0.0000000 | 0.0000000 |
| juvenil          | 0.2857143        | 0.5000000 | 0.0000000 |
| adulto           | 0.4285714        | 0.3333333 | 0.6666667 |

\$F

|                  | plantas_pequeñas | juvenil   | adulto    |
|------------------|------------------|-----------|-----------|
| plantas_pequeñas | 0.0000000        | 0.0000000 | 0.1666667 |
| juvenil          | 0.0000000        | 0.0000000 | 0.0000000 |
| adulto           | 0.0000000        | 0.0000000 | 0.0000000 |

## 6.7 Unión de matrices

Si deseas obtener una matriz que combine tanto las transiciones como la fertilidad, puedes utilizar el argumento **TF** = FALSE en relación con **matA**.

En este caso, la matriz resultante se define como:

$$matA = matT + matF$$

donde:

- `matT` es la matriz de transiciones.
- `matF` es la matriz de fertilidad.

```
projection.matrix(Orchis_ficticia_df, stage, fate, fertilidad, stages, TF=FALSE)
```

|                  | plantas_pequeñas | juvenil   | adulto    |
|------------------|------------------|-----------|-----------|
| plantas_pequeñas | 0.1428571        | 0.0000000 | 0.1666667 |
| juvenil          | 0.2857143        | 0.5000000 | 0.0000000 |
| adulto           | 0.4285714        | 0.3333333 | 0.6666667 |

---

## 6.8 Matriz de Clonaje

Muchas orquídeas se reproducen por clonación, lo que significa que los individuos son genéticamente idénticos y se originan a partir de la planta madre. En algunos casos, estos clones se separan mediante un keiki o por división de los rizomas cuando estos se deterioran o mueren. Este es un método de reproducción asexual.

En este contexto, la matriz de transiciones puede modificarse para incluir el clonaje. Existen pocos ejemplos en orquídeas donde se incorpora una submatriz de clonaje, denominada `matC`. Desde el punto de vista evolutivo, este proceso representa el crecimiento de un individuo y no debe considerarse como fecundidad, ya que es un mecanismo distinto. Este concepto de crecimiento es relevante en los análisis de selección natural y deriva genética, dos procesos evolutivos fundamentales. Además, la reproducción asexual puede influir en el crecimiento intrínseco y la dinámica poblacional.

Como ejemplo, revisamos el trabajo sobre *Cypripedium calceolus* en Europa (García, Goni & Guzmán, 2010). En este estudio, los autores analizaron la dinámica de poblaciones situadas en la periferia de su distribución geográfica, específicamente en los Pirineos y el norte de Europa. Aprovecharemos este caso para mostrar cómo se identifican especies que incluyen una matriz de clonaje (**matC**).

El primer paso consiste en descargar los datos desde la base de datos COMPADRE utilizando la función `cdb_fetch` del paquete **Rcompadre**. Esta función obtiene la información y la almacena en un objeto llamado `compadre`.

```
compadre <- readRDS("data/COMPADRE_2026.rds") # Usar este código para bajar los datos de
```

---

### 6.8.1 Extraer de COMPADRE las matrices *Cypripedium calceolus*.

Segundo extraer de la base de datos de COMPADRE las matrices de transiciones de *Cypripedium calceolus* del artículo mencionado arriba.

Los pasos son los siguientes

- Filtrar la base de datos para *Cypripedium calceolus*
- Filtrar la base de datos para el DOI del artículo
- Filtrar la base de datos para la dimensión de la matriz de transiciones (6x6)
- Filtrar la base base de datos para una matriz en particular (242623)

NOTA que se podría haber filtrado parte la **MatrixID** solamente si uno conoce la identificación de esta matriz en la base de datos.

```
Cyp_cal_matC = compadre %>%
  select(MatrixID, SpeciesAccepted, mat, Authors, DOI_ISBN, MatrixDimension) %>%
  filter(
    SpeciesAccepted %in% c("Cypripedium calceolus"),
    DOI_ISBN %in% c("10.1111/j.1523-1739.2010.01466.x"),
    MatrixDimension %in% c(6),
    MatrixID %in% c(242623)
  ) # filtrar la base de datos para la matriz de interés

#Cyp cal matC$mat # para el año 2000 a 2001
```

### 6.8.2 matA de *Cypripedium calceolus*, MatrixID = 242623

Las matriz de transiciones de *Cypripedium calceolus* tiene una matriz de transiciones de fecundidad y de clonaje,  $\text{matA}=\text{matU}+\text{matF}+\text{matC}$

```
matA <- matA(Cyp_cal_matC) # extraer la matriz a partir de la base de datos de COMPADRE
matcc=as.matrix(as.data.frame(matA)) # convertir la matriz a un data frame y luego a un
matcc
```

|    | A1   | A2   | A3   | A4   | A5   | A6   |
|----|------|------|------|------|------|------|
| A1 | 0.11 | 0.10 | 0.22 | 0.14 | 0.22 | 0.17 |
| A2 | 0.17 | 0.52 | 0.02 | 0.01 | 0.02 | 0.03 |
| A3 | 0.31 | 0.19 | 0.47 | 0.20 | 0.05 | 0.07 |
| A4 | 0.11 | 0.10 | 0.16 | 0.35 | 0.13 | 0.09 |
| A5 | 0.31 | 0.00 | 0.04 | 0.22 | 0.57 | 0.41 |
| A6 | 0.00 | 0.00 | 0.00 | 0.00 | 0.00 | 0.27 |

## 6.9 Extraer la matriz de Clonaje

NOTE que la matriz de clonaje es la siguiente.

Lo que se observa es que hay NUEVOS INDIVIDUOS en el tiempo  $t_{i+1}$ , que provienen de los individuos en el tiempo  $t_i$ . En este caso, son individuos de provienen de a las etapas 4, 5 y 6 que producen individuos nuevos (por clonaje) en la etapa 3 y 4.

```
matC <- matC(Cyp_cal_matC) # extraer la matriz de clonaje a partir de la base de datos d
matC
```

```
[[1]]
      C1 C2 C3  C4   C5   C6
C1  0  0  0 0.0 0.00 0.00
C2  0  0  0 0.0 0.00 0.00
C3  0  0  0 0.2 0.05 0.07
C4  0  0  0 0.0 0.00 0.09
C5  0  0  0 0.0 0.00 0.00
C6  0  0  0 0.0 0.00 0.00
```

### 6.9.1 El ciclo de vida de *Cypridium calceolus* con clonaje

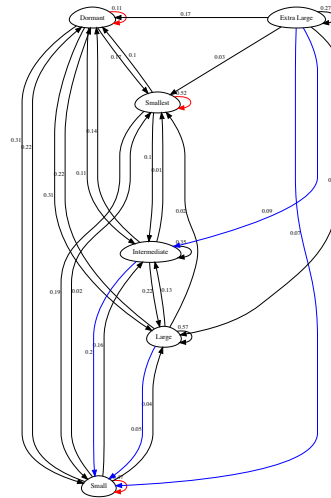
Ahora vemos el ciclo de vida de *Cypridium calceolus* en este estudio donde las lineas azules son las transiciones de una etapa a otra por clonaje.

```
#etapas <- c("A1", "A2", "A3", "A4", "A5", "A6")
etapas <- c("Dormant", "Smallest", "Small", "Intermediate", "Large", "Extra Large") # Et
etapas <- factor(etapas, levels = c("Dormant", "Smallest", "Small", "Intermediate", "La
title <- NULL
graph <- expand.grid(to = etapas, from = etapas)
graph$trans <- round(c(matcc), 2)
graph <- graph[graph$trans > 0, ]
nodes <- paste(paste0("", etapas, ""), collapse = "; ")
graph$min_len <- (as.numeric(graph$to) - as.numeric(graph$from)) * 3
graph$col <- c("red", "black", "black", "black", "black",
              "black", "red", "black", "black",
              "black", "black", "red", "black", "black",
              "black", "black", "blue", "black", "black",
              "black", "black", "blue", "black", "black",
              "black", "black", "blue", "blue", "black", "black"
)
edges <- paste0("", graph$from, "", " -> ", "", graph$to, "",
               "[minlen=", graph$min_len,
```

```

",fontsize=", 10,
",color=", graph$col,
",xlabel=", paste("\n", graph$trans),
"\n"]\n",
collapse = ""
)
grViz(
  paste(
    "
digraph {
  {
    graph[overlap=false];
    rank=etapas;
    node [shape=", "egg", ", fontsize=", 12, "];",
    nodes, "
  }",
    "ordering=out
x [style=invis]
x -> {", nodes, "} [style=invis]", edges,
    "labelloc=\"t\";
label=\"", title, "\""
}"
)
)

```



Los estudios sobre orquídeas que incorporan matrices de clonaje son poco frecuentes, pero es fundamental considerar este proceso en los análisis de dinámica poblacional. El clonaje

puede tener un impacto significativo tanto en la dinámica de las poblaciones como en la evolución de las especies.

En este contexto, el clonaje se entiende como una forma de reproducción asexual que puede modificar la estructura genética de la población y afectar su capacidad de adaptación frente a cambios ambientales.

---

### 6.9.2 Limitaciones de los métodos tradicionales

El uso de métodos tradicionales, ya sea mediante cálculos manuales o utilizando la función `projection.matrix` del paquete `popbio`, puede resultar problemático. Las dificultades surgen principalmente por condiciones de muestreo, especialmente cuando el tamaño de muestra es pequeño o cuando las transiciones son naturalmente raras (Tremblay et al., 2021; Gascoigne et al., 2023), situación común en especies poco frecuentes.

En el capítulo **Impacto de datos sin sentido biológico** se analizan los problemas que ocurren cuando los datos carecen de coherencia biológica, cómo construir matrices más confiables y cuándo es apropiado emplear el método bayesiano en los análisis. Este enfoque permite resolver inconsistencias y garantizar que las matrices sean más compatibles con la biología de la especie estudiada, aumentando así la relevancia de los resultados.

Los autores recomiendan utilizar el **método bayesiano** para el análisis de matrices de transición en la mayoría de los casos, excepto cuando el tamaño de muestra es muy grande y ninguna transición es rara. El método bayesiano es un enfoque estadístico que permite incorporar información previa y ajustar los modelos a los datos observados. Este enfoque es especialmente útil cuando se trabaja con datos escasos o transiciones poco frecuentes. Aunque su implementación puede ser más compleja, ofrece mayor flexibilidad y robustez, además de estimar la confianza en los valores de transición y fecundidad.

